



Министерство науки и высшего образования Российской
Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И УПРАВЛЕНИЕ

КАФЕДРА УПРАВЛЕНИЕ В ТЕХНИЧЕСКИХ СИСТЕМАХ

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ НА ТЕМУ:

*Разработка и реализация системы автоматической
генерации технических учебных задач с модулем
логического рассуждения (Reasoning) на основе
претренированной языковой модели*

Студент группы ИУК3-72Б

(подпись, дата)

М.О. Сухацкий

(И.О. Фамилия)

Руководитель курсового
проекта

(подпись, дата)

М.О. Корлякова

(И.О. Фамилия)

Калуга, 2025

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой ИУКЗ
(Д.И. Мельников)
« » 2025г.

З А Д А Н И Е на выполнение курсового проекта

по дисциплине Статистическая динамика информационно-управляющих систем

Студент группы ИУКЗ-72Б Сухацкий Максим Олегович
(фамилия, имя, отчество)

Тема курсового проекта Разработка и реализация системы автоматической генерации технических учебных задач с модулем логического рассуждения (Reasoning) на основе претренированной языковой модели

Направленность КП учебный

Источник тематики кафедра ИУКЗ

Задание

Выбрать подходящую архитектуру модели

Собрать данные необходимые для дообучения модели

Провести тестирование модели в live-режиме

Подготовить необходимые интерфейсы для удобства обращения с моделью.

Оформление курсового проекта

Расчетно-пояснительная записка на листах формата А4.

Дата выдачи задания « » 2025 г.

Руководитель

(подпись, дата)

М.О. Корлякова

(И.О. Фамилия)

Студент

(подпись, дата)

М.О. Сухацкий

(И.О. Фамилия)

**Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)**

КАЛЕНДАРНЫЙ ПЛАН на выполнение курсового проекта

по дисциплине **Статистическая динамика информационно-управляющих систем**

Студент группы **ИУКЗ – 72Б Сухацкий Максим Олегович**
(фамилия, имя, отчество)

Тема курсового проекта **Проектирование технологической оснастки заготовительного производства**

№	Наименование этапов	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель	Куратор
1	Задание на выполнение	1-я нед.			
2	Выполнение основных расчетов и эскизов графической части	10-я нед.			
3	Выполнение и окончательное оформление графической части и расчетно-пояснительной записки	14-я нед.			
4	Защита	17-я нед.			

Студент _____ г. Руководитель _____ г.
(подпись, дата) (подпись, дата)

СОДЕРЖАНИЕ

Оглавление

ВВЕДЕНИЕ	6
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ ОБУЧЕНИЯ	8
1.1. Концепция интеллектуальных систем обучения	8
1.2. Сократический метод в образовании	9
1.3. Байесовское отслеживание знаний (ВКТ)	10
1.4. Статический анализ программного кода	10
ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ MITS	12
2.1. Архитектура системы	12
2.2. Модуль сократического тьютора	12
2.3. Модуль отслеживания знаний	13
2.4. Модуль выполнения и анализа кода	14
ГЛАВА 3. РЕАЛИЗАЦИЯ СИСТЕМЫ	16
3.1. Технологический стек	16
3.2. Банк алгоритмических задач	16
3.3. Пользовательский интерфейс	17
ГЛАВА 4. ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА	18
4.1. Метрики оценки эффективности	18
4.2. Результаты тестирования	18
ЗАКЛЮЧЕНИЕ	20
СПИСОК ЛИТЕРАТУРЫ	21
ПРИЛОЖЕНИЕ А	23
Исходный код модуля сократического тьютора (фрагменты)	23
ПРИЛОЖЕНИЕ Б	25
Исходный код модуля выполнения кода (фрагменты)	25

ВВЕДЕНИЕ

Актуальность исследования

В современном образовании наблюдается устойчивый рост интереса к персонализированному обучению, которое учитывает индивидуальные особенности каждого студента. Традиционные методы обучения, основанные на лекционном формате, не всегда способны обеспечить необходимый уровень индивидуализации и оперативную обратную связь [1]. Особенно остро эта проблема проявляется в области обучения математике и программированию, где понимание концепций требует активного участия студента в решении задач.

Интеллектуальные системы обучения (Intelligent Tutoring Systems, ITS) представляют собой перспективное направление в образовательных технологиях, позволяющее автоматизировать процесс персонализированного обучения [2]. Современные достижения в области больших языковых моделей (LLM) открывают новые возможности для создания адаптивных систем, способных вести естественный диалог со студентом и применять педагогические стратегии, ранее доступные только опытным преподавателям [3].

Сократический метод обучения, основанный на направлении студента к самостоятельному открытию знаний через систему наводящих вопросов, признаётся одним из наиболее эффективных педагогических подходов [4]. Однако его применение требует высокой квалификации преподавателя и значительных временных затрат, что делает масштабирование затруднительным. Автоматизация сократического метода с использованием современных технологий искусственного интеллекта является актуальной научно-практической задачей.

Цель работы

Целью данной курсовой работы является разработка интеллектуальной системы обучения MITS (Mathematical Intelligent Tutoring System), реализующей сократический метод для обучения математике и программированию с использованием большой языковой модели и адаптивного отслеживания знаний студента.

Задачи исследования

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) провести анализ существующих подходов к созданию интеллектуальных систем обучения и методов отслеживания знаний;
- 2) разработать архитектуру системы, включающую модули сократического тьютора, отслеживания знаний и выполнения кода;
- 3) реализовать сократического агента-тьютора на основе большой языковой модели с системой педагогических стратегий;
- 4) реализовать модуль Bayesian Knowledge Tracing для адаптивного отслеживания уровня владения навыками;
- 5) разработать безопасную песочницу для выполнения программного кода с анализом ошибок;
- 6) создать банк алгоритмических задач с тестами и системой подсказок;
- 7) провести экспериментальную оценку эффективности системы.

Научная новизна

Научная новизна работы заключается в комплексной интеграции сократического метода обучения с байесовским отслеживанием знаний и статическим анализом кода в единую систему, а также в разработке системы педагогических стратегий для агента-тьютора, предотвращающей преждевременное раскрытие ответов.

Практическая значимость

Практическая значимость работы состоит в создании работоспособной системы обучения, которая может использоваться для самостоятельной подготовки студентов к экзаменам по программированию и математике, а также в качестве дополнительного инструмента в образовательном процессе.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ ОБУЧЕНИЯ

1.1. Концепция интеллектуальных систем обучения

Интеллектуальные системы обучения (Intelligent Tutoring Systems, ITS) представляют собой класс программных систем, использующих методы искусственного интеллекта для обеспечения персонализированного обучения [5]. В отличие от традиционных компьютерных обучающих систем, ITS способны адаптировать учебный материал и стратегии обучения под индивидуальные потребности каждого студента.

Классическая архитектура ITS включает четыре основных компонента [6]:

- модель предметной области (domain model), содержащая экспертные знания по изучаемой теме;
- модель студента (student model), отслеживающая текущий уровень знаний и навыков обучаемого;
- педагогический модуль (pedagogical module), определяющий стратегию обучения;
- интерфейс пользователя (user interface), обеспечивающий взаимодействие со студентом.

Исследования показывают, что эффективность индивидуального обучения с репетитором может превышать эффективность классного обучения на два стандартных отклонения (так называемая «проблема двух сигм» Блума) [7]. Задача ITS — приблизиться к этому уровню эффективности при помощи автоматизации.

Современные ITS можно классифицировать по нескольким критериям [8]:

- по предметной области: математика, программирование, естественные науки, языки;
- по методу взаимодействия: диалоговые, основанные на задачах, гибридные;
- по технологии ИИ: правила, нейронные сети, большие языковые модели.

Появление больших языковых моделей (LLM), таких как GPT-4 и Qwen3, открыло новые возможности для создания диалоговых ITS [9]. Эти модели способны

генерировать естественные ответы, понимать контекст разговора и применять различные педагогические стратегии. Однако использование LLM в образовании требует решения специфических проблем: предотвращение «галлюцинаций», контроль качества генерируемого контента и обеспечение педагогической корректности ответов.

1.2. Сократический метод в образовании

Сократический метод, названный в честь древнегреческого философа Сократа, представляет собой форму диалога, основанную на задавании вопросов для стимулирования критического мышления и раскрытия идей [10]. В образовательном контексте этот метод противопоставляется дидактическому подходу, при котором знания передаются непосредственно от учителя к ученику.

Ключевые принципы сократического метода в обучении [11]:

- студент является активным участником процесса познания, а не пассивным получателем информации;
- учитель направляет через вопросы, но не даёт готовых ответов;
- ошибки рассматриваются как возможности для обучения, а не как неудачи;
- цель — развитие глубокого понимания, а не механического запоминания.

В работе Wang et al. [12] представлена система SocraticLLM, определяющая пять педагогических стратегий для сократического тьютора:

- 1) Scaffolding (каркас) — построение структуры для мышления студента через направляющие вопросы;
- 2) Problematize (проблематизация) — стимулирование рефлексии через вопросы «почему?» и «что если?»;
- 3) Rectify (исправление) — мягкое указание на ошибки без прямой критики;
- 4) Encourage (поддержка) — положительное подкрепление правильных шагов;
- 5) Tell (рассказ) — прямое предоставление информации как крайняя мера.

Исследования подтверждают эффективность сократического метода для развития навыков решения проблем и критического мышления [13]. Однако его

применение требует от преподавателя умения задавать правильные вопросы в нужный момент, что является сложной задачей для автоматизации.

1.3. Байесовское отслеживание знаний (ВКТ)

Байесовское отслеживание знаний (Bayesian Knowledge Tracing, ВКТ) — это вероятностная модель для оценки уровня владения студентом отдельными навыками на основе истории его ответов [14]. Модель основана на скрытых марковских процессах и позволяет обновлять оценку знаний после каждого взаимодействия со студентом.

Модель ВКТ описывается четырьмя параметрами для каждого навыка:

- $P(L_0)$ — начальная вероятность владения навыком до первой попытки;
- $P(T)$ — вероятность перехода к владению навыком после одной попытки (вероятность обучения);
- $P(G)$ — вероятность правильного ответа при отсутствии навыка (вероятность угадывания);
- $P(S)$ — вероятность неправильного ответа при владении навыком (вероятность ошибки по невнимательности).

Обновление оценки владения навыком $P(L_n)$ после наблюдения результата выполняется по формуле Байеса. При правильном ответе:

$$P(L_n | \text{correct}) = P(L_{n-1}) \times (1 - P(S)) / P(\text{correct})$$

$$\text{где } P(\text{correct}) = P(L_{n-1}) \times (1 - P(S)) + (1 - P(L_{n-1})) \times P(G)$$

После обновления апостериорной вероятности применяется правило обучения:

$$P(L_n) = P(L_n | \text{obs}) + (1 - P(L_n | \text{obs})) \times P(T)$$

ВКТ позволяет решать несколько важных задач в ITS [15]: определение момента освоения навыка (mastery detection), предсказание успешности на следующей задаче, адаптивный выбор задач оптимальной сложности. В отличие от простого подсчёта правильных ответов, ВКТ учитывает вероятностную природу оценки знаний и позволяет строить иерархии связанных навыков.

1.4. Статический анализ программного кода

Статический анализ кода — это метод анализа программного обеспечения без его фактического выполнения [16]. В контексте систем обучения программированию статический анализ позволяет выявлять стилистические проблемы, потенциальные ошибки и неоптимальные паттерны в коде студента.

Для языка Python основным инструментом статического анализа является модуль AST (Abstract Syntax Tree), предоставляющий возможность разбора исходного кода в древовидную структуру [17]. Анализ AST позволяет:

- обнаруживать синтаксические ошибки до выполнения;
- находить неиспользуемые переменные и импорты;
- выявлять антипаттерны (например, слишком широкий excerpt);
- оценивать цикломатическую сложность кода;
- определять временную и пространственную сложность алгоритма.

Цикломатическая сложность, предложенная МакКейбом [18], измеряет количество линейно независимых путей в программе и вычисляется как:

$$V(G) = E - N + 2P$$

где E — число рёбер графа потока управления, N — число вершин, P — число компонент связности.

В контексте обучения программированию статический анализ дополняет динамическое тестирование, позволяя предоставлять студенту обратную связь о качестве кода, а не только о его корректности [19].

ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ MITS

2.1. Архитектура системы

Система MITS (Mathematical Intelligent Tutoring System) спроектирована на основе модульной архитектуры, обеспечивающей разделение ответственности между компонентами и возможность независимого развития каждого модуля. Общая архитектура системы представлена на рисунке 1 и включает следующие основные компоненты:

- модуль LLM Client для взаимодействия с большой языковой моделью;
- агент SocraticTutor, реализующий педагогические стратегии;
- модуль KnowledgeTracker для отслеживания знаний студента;
- модуль CodeExecutor для безопасного выполнения кода;
- модуль CodeAnalyzer для статического анализа;
- банки задач по математике и алгоритмам;
- веб-интерфейс на базе Gradio.

В качестве большой языковой модели выбрана Qwen3-30B-A3B-Instruct, работающая через сервер Ollama. Выбор обусловлен несколькими факторами: модель использует архитектуру Mixture of Experts (MoE), активируя только 3 миллиарда параметров при общем размере 30 миллиардов, что обеспечивает баланс между качеством генерации и вычислительной эффективностью. Модель поддерживает режим «thinking», позволяющий генерировать внутренние рассуждения перед формированием ответа.

Взаимодействие между компонентами осуществляется через определённые интерфейсы. LLM Client предоставляет методы для синхронной и потоковой генерации ответов с поддержкой режима thinking и JSON-режима для структурированного вывода. Агент-тьютор получает от клиента структурированные ответы с указанием педагогической стратегии (move) и текста сообщения.

2.2. Модуль сократического тьютора

Модуль SocraticTutorAgent является центральным компонентом педагогической логики системы. Он реализует сократический метод обучения,

направляя студента к самостоятельному решению через систему наводящих вопросов. Структура модуля представлена в таблице 1.

Таблица 1 — Педагогические стратегии агента-тьютора

Стратегия	Триггер применения	Пример действия
Scaffolding	Студент начинает работу над задачей	«С чего бы вы начали? Какой первый шаг?»
Problematize	Студент задаёт вопрос	«Почему вы так думаете? А что если...?»
Rectify	Студент допустил ошибку	«Давайте проверим этот шаг ещё раз»
Encourage	Студент на верном пути	«Отличная мысль! Продолжайте»
Hint	Много неудачных попыток	Наводящий вопрос из банка подсказок
Tell	Все подсказки исчерпаны	Раскрытие решения (крайняя мера)

Важным аспектом реализации является механизм предотвращения преждевременного раскрытия ответа. Система выполняет проверку генерируемых сообщений на наличие прямого ответа или решения задачи. При обнаружении такого содержимого применяется санитизация: сообщение заменяется на безопасный scaffolding-ответ.

Выбор стратегии осуществляется на основе анализа текущей ситуации, включающей: количество попыток студента, число использованных подсказок, эмоциональное состояние (определяется по ключевым словам), наличие вопроса в сообщении, результат верификации ответа. Приоритет стратегий реализован в виде последовательности условий, начиная с проверки успешного решения (encourage) и заканчивая scaffolding как стратегией по умолчанию.

2.3. Модуль отслеживания знаний

Модуль KnowledgeTracker реализует Bayesian Knowledge Tracing для персонализированной оценки уровня владения навыками каждого студента. Модель студента (StudentModel) хранит состояния для более чем 40 навыков, организованных в иерархию с учётом зависимостей (prerequisites). Основные категории навыков представлены в таблице 2.

Таблица 2 — Иерархия навыков в системе MITs

Категория	Навыки	Prerequisites
Базовая алгебра	arithmetic, fractions, linear_equations	—
Функции	functions_basics, function_composition	linear_equations
Пределы и производные	limits, derivatives_basic, chain_rule	functions_basics
Интегралы	integrals_basic, integration_by_parts	derivatives_basic
Программирование	variables, loops, functions_prog, recursion	последовательные
Алгоритмы	data_structures, algorithms	recursion, loops

Каждый навык характеризуется состоянием SkillState, включающим: текущую оценку владения (mastery), количество попыток и успехов, дату последней практики, а также индивидуальные параметры ВКТ. Значения по умолчанию: $P(L_0) = 0.3$, $P(T) = 0.1$, $P(G) = 0.2$, $P(S) = 0.1$.

Модуль предоставляет следующие ключевые функции: обновление mastery после каждой попытки; определение наиболее слабых и сильных навыков; рекомендация следующего навыка для изучения на основе prerequisites; предсказание вероятности успеха на задаче; рекомендация сложности задачи на основе среднего mastery.

2.4. Модуль выполнения и анализа кода

Модуль CodeExecutor обеспечивает безопасное выполнение Python-кода студентов в изолированной среде (песочнице). Ключевые требования к модулю: ограничение времени выполнения (по умолчанию 5 секунд), блокировка опасных операций, захват stdout/stderr, информативный анализ ошибок для обратной связи.

Безопасность обеспечивается классом CodeSecurityChecker, выполняющим статический анализ AST перед выполнением. Блокируются следующие категории:

- системные модули: os, sys, subprocess, socket;
- сетевые модули: requests, urllib, http;
- опасные встроенные функции: exec, eval, compile, open, __import__.

Класс ErrorAnalyzer обеспечивает дружественный анализ ошибок выполнения. Для каждого типа исключения (ZeroDivisionError, IndexError, TypeError и др.) предоставляется: локализованное описание ошибки, подсказка по

исправлению, пример правильного кода. Это позволяет студенту понять причину ошибки и способ её устранения без раскрытия решения задачи.

Модуль `CodeAnalyzer` дополняет выполнение статическим анализом, выявляя: стилистические проблемы (`range(len())` вместо `enumerate`), потенциальные ошибки (мутабельные аргументы по умолчанию), неоптимальные паттерны (использование `global`). Анализ сложности определяет временную и пространственную сложность на основе структуры циклов и рекурсии.

ГЛАВА 3. РЕАЛИЗАЦИЯ СИСТЕМЫ

3.1. Технологический стек

Система MITS реализована на языке Python версии 3.12 с использованием современных библиотек и фреймворков. Основные компоненты технологического стека представлены в таблице 3.

Таблица 3 — Технологический стек системы MITS

Компонент	Технология	Назначение
Язык программирования	Python 3.12+	Основной язык разработки
LLM Backend	Ollama + Qwen3-30B-A3B	Генерация ответов тьютора
Веб-интерфейс	Gradio 4.x	UI с поддержкой LaTeX
Математика	SymPy	Символьные вычисления
Статический анализ	ast (стандартная библиотека)	Анализ Python-кода
Логирование	structlog	Структурированные логи

Выбор Gradio в качестве фреймворка для веб-интерфейса обусловлен встроенной поддержкой рендеринга LaTeX-формул, что критически важно для отображения математических задач и решений. Интерфейс поддерживает как inline-формулы ($\$...\$$), так и display-формулы ($\$...\$$).

Ollama используется как локальный сервер для запуска LLM, обеспечивая независимость от облачных сервисов и контроль над конфиденциальностью данных студентов. Модель Qwen3-30B-A3B загружается командой: `ollama pull qwen3:30b-a3b-instruct-2507`.

3.2. Банк алгоритмических задач

Система включает банк из более чем 50 алгоритмических задач трёх уровней сложности, охватывающих основные категории алгоритмов. Структура банка задач представлена в таблице 4.

Таблица 4 — Структура банка алгоритмических задач

Категория	Количество	Примеры задач
Массивы (arrays)	12	Two Sum, Maximum Subarray, Merge Intervals
Строки (strings)	8	Palindrome, Valid Parentheses, Anagram
Поиск (searching)	6	Binary Search, First Bad Version
Сортировка (sorting)	5	Merge Sort, Quick Sort
Рекурсия (recursion)	6	Fibonacci, Factorial, Tower of Hanoi

Категория	Количество	Примеры задач
ДП (dynamic)	10	LCS, Edit Distance, Knapsack
Графы (graphs)	5	Number of Islands, Shortest Path

Каждая задача в банке содержит: уникальный идентификатор, заголовок на русском и английском языках, описание условия, формат входных и выходных данных, ограничения (constraints), примеры с пояснениями, скрытые тесты для проверки, подсказки для сократического обучения, эталонное решение с объяснением.

Задачи распределены по сложности: Easy (40%), Medium (40%), Hard (20%). Сложность определяется несколькими факторами: количество необходимых концепций, требуемая алгоритмическая оптимизация, типичное время решения для опытного программиста.

3.3. Пользовательский интерфейс

Пользовательский интерфейс системы реализован как веб-приложение с четырьмя основными вкладками: свободный диалог с AI, математические задачи, алгоритмические задачи, профиль знаний студента.

Вкладка свободного диалога позволяет студенту задавать любые вопросы по математике и программированию. Система поддерживает контекст разговора, сохраняя последние 10 сообщений для формирования истории диалога.

Вкладка алгоритмических задач предоставляет: выбор задачи по сложности и категории, редактор кода с подсветкой синтаксиса, кнопку быстрого тестирования на примерах, кнопку полной отправки на скрытые тесты, анализ кода с оценкой сложности, систему подсказок.

Интерфейс запускается на локальном сервере (<http://localhost:7860>) и доступен через веб-браузер. Система поддерживает горячую перезагрузку при изменении кода, что удобно для разработки и отладки.

ГЛАВА 4. ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА

4.1. Метрики оценки эффективности

Для оценки эффективности интеллектуальных систем обучения используется ряд специализированных метрик, позволяющих количественно измерить качество педагогического взаимодействия [20]. В системе MITS реализованы следующие метрики:

Success@K — доля сессий, в которых студент успешно решил задачу за не более чем K попыток. Метрика отражает эффективность направляющей помощи системы. Целевое значение $\text{Success@10} > 60\%$.

Telling@K — доля сессий, в которых система была вынуждена показать ответ до K -й попытки. Метрика характеризует способность системы направлять без раскрытия решения. Целевое значение $\text{Telling@10} < 15\%$.

Hint Efficiency — среднее количество подсказок, использованных до успешного решения. Низкое значение указывает на эффективность каждой подсказки. Целевое значение < 2.5 .

KT AUC-ROC — площадь под ROC-кривой для предсказания успешности студента модулем Knowledge Tracing. Отражает точность модели студента. Целевое значение > 0.75 .

Таблица 5 — Целевые значения метрик эффективности

Метрика	Целевое значение	Описание
Success@10	$> 60\%$	Задач решено за ≤ 10 попыток
Telling@10	$< 15\%$	Сессий с показом ответа
Hint Efficiency	< 2.5	Среднее подсказок до успеха
KT AUC-ROC	> 0.75	Точность предсказания успеха

4.2. Результаты тестирования

Тестирование системы проводилось на выборке из 25 сессий с решением алгоритмических задач различной сложности. Результаты представлены в таблице 6.

Таблица 6 — Результаты экспериментальной оценки

Показатель	Значение	Целевое
Success@10	68%	> 60% ✓
Telling@10	12%	< 15% ✓
Hint Efficiency	2.3	< 2.5 ✓
Среднее время сессии, мин	8.5	—
Задач Easy решено, %	85%	—
Задач Medium решено, %	62%	—
Задач Hard решено, %	45%	—

Результаты демонстрируют, что система достигает целевых значений по всем основным метрикам. Показатель Success@10 = 68% превышает целевое значение 60%, что свидетельствует об эффективности сократического подхода в направлении студентов к решению.

Низкое значение Telling@10 = 12% подтверждает, что система успешно избегает преждевременного раскрытия ответов, сохраняя принципы сократического метода. Механизм санитизации ответов и постепенное нарастание подсказок работают корректно.

Анализ результатов по сложности задач показывает ожидаемое снижение успешности с ростом сложности: Easy (85%), Medium (62%), Hard (45%). Это соответствует распределению, характерному для образовательных платформ типа LeetCode.

Модуль Knowledge Tracing продемонстрировал способность адаптировать рекомендации сложности на основе истории взаимодействий. После 5-7 сессий модель студента стабилизируется и начинает давать релевантные рекомендации по следующим навыкам для изучения.

ЗАКЛЮЧЕНИЕ

В рамках данной курсовой работы была разработана интеллектуальная система обучения MITS (Mathematical Intelligent Tutoring System), реализующая сократический метод для обучения математике и программированию.

В ходе работы были выполнены все поставленные задачи:

1) Проведён анализ существующих подходов к созданию ITS, рассмотрены методы сократического обучения, Bayesian Knowledge Tracing и статического анализа кода.

2) Разработана модульная архитектура системы, включающая компоненты LLM-клиента, сократического тьютора, отслеживания знаний и выполнения кода.

3) Реализован агент SocraticTutorAgent с шестью педагогическими стратегиями и механизмом предотвращения раскрытия ответов.

4) Реализован модуль Bayesian Knowledge Tracing с иерархией из 40+ навыков и функциями адаптивных рекомендаций.

5) Разработана безопасная песочница CodeExecutor с блокировкой опасных операций и информативным анализом ошибок.

6) Создан банк из 50+ алгоритмических задач с тестами, подсказками и эталонными решениями.

7) Проведена экспериментальная оценка, подтвердившая достижение целевых метрик: $\text{Success}@10 = 68\%$, $\text{Telling}@10 = 12\%$.

Научная значимость работы состоит в комплексной интеграции сократического метода с байесовским отслеживанием знаний и статическим анализом кода. Практическая значимость заключается в создании работоспособной системы, которая может использоваться для самостоятельной подготовки студентов.

Направления дальнейшего развития системы включают: fine-tuning языковой модели на образовательных диалогах, интеграцию RAG с документацией и учебными материалами, добавление визуализации алгоритмов, мультимодальность (OCR для рукописных решений), self-improvement loop для автоматического улучшения педагогических стратегий на основе результатов сессий.

СПИСОК ЛИТЕРАТУРЫ

1. Koedinger, K. R., Anderson, J. R., Hadley, W. H., & Mark, M. A. Intelligent tutoring goes to school in the big city // *International Journal of Artificial Intelligence in Education*. — 1997. — Vol. 8. — P. 30-43.
2. Nwana, H. S. Intelligent Tutoring Systems: An Overview // *Artificial Intelligence Review*. — 1990. — Vol. 4. — P. 251-277.
3. OpenAI. GPT-4 Technical Report // arXiv preprint arXiv:2303.08774. — 2023.
4. Paul, R., & Elder, L. *The Thinker's Guide to The Art of Socratic Questioning*. — Foundation for Critical Thinking, 2006.
5. VanLehn, K. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems // *Educational Psychologist*. — 2011. — Vol. 46(4). — P. 197-221.
6. Woolf, B. P. *Building Intelligent Interactive Tutors*. — Morgan Kaufmann, 2009.
7. Bloom, B. S. The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring // *Educational Researcher*. — 1984. — Vol. 13(6). — P. 4-16.
8. Graesser, A. C., Conley, M. W., & Olney, A. Intelligent tutoring systems // *APA Educational Psychology Handbook*. — 2012. — Vol. 3. — P. 451-473.
9. Kasneci, E., et al. ChatGPT for good? On opportunities and challenges of large language models for education // *Learning and Individual Differences*. — 2023. — Vol. 103.
10. Vlastos, G. *Socratic Studies*. — Cambridge University Press, 1994.
11. Reich, R. The Socratic method: What it is and how to use it in the classroom // *Speaking of Teaching*. — 2003. — Vol. 13(1). — P. 1-4.
12. Wang, R., et al. SocraticLLM: Teaching LLMs the Socratic Method // arXiv preprint arXiv:2310.09825. — 2023.
13. Boghossian, P. Socratic Pedagogy: Perplexity, humiliation, shame and a broken egg // *Educational Philosophy and Theory*. — 2012. — Vol. 44(7). — P. 710-720.

14. Corbett, A. T., & Anderson, J. R. Knowledge tracing: Modeling the acquisition of procedural knowledge // User Modeling and User-Adapted Interaction. — 1994. — Vol. 4(4). — P. 253-278.
15. Pelánek, R. Bayesian knowledge tracing, logistic models, and beyond: An overview of learner modeling techniques // User Modeling and User-Adapted Interaction. — 2017. — Vol. 27. — P. 313-350.
16. Ayewah, N., et al. Using Static Analysis to Find Bugs // IEEE Software. — 2008. — Vol. 25(5). — P. 22-29.
17. Python Software Foundation. ast — Abstract Syntax Trees // Python Documentation. — 2024.
18. McCabe, T. J. A Complexity Measure // IEEE Transactions on Software Engineering. — 1976. — Vol. SE-2(4). — P. 308-320.
19. Keuning, H., Jeuring, J., & Heeren, B. A systematic literature review of automated feedback generation for programming exercises // ACM Transactions on Computing Education. — 2019. — Vol. 19(1).
20. Rus, V., Stefanescu, D., Niraula, N., & Graesser, A. C. DeepTutor: Towards macro- and micro-adaptive conversational intelligent tutoring at scale // Proceedings of the 2014 ACM Conference on L@S. — 2014. — P. 209-210.

Основной код можно посмотреть на [github:](https://github.com/Siesher/MIST)
<https://github.com/Siesher/MIST>

Там же приведена инструкция по запуску проекта

ПРИЛОЖЕНИЕ А

Исходный код модуля сократического тьютора (фрагменты)

Класс `SocraticTutorAgent` — основной агент-тьютор:

```
class SocraticTutorAgent(BaseAgent):
    """
    Main tutoring agent implementing the Socratic method.

    Teaching Structure:
    1. Review - Understand student's current state
    2. Guidance/Heuristic - Lead with questions
    3. Rectification - Correct misconceptions gently
    4. Summarization - Reinforce learning after success
    """

    def __init__(self, llm_client: LLMClient):
        super().__init__(
            name="SocraticTutor",
            llm_client=llm_client,
            system_prompt=SOCRATIC_TUTOR_SYSTEM
        )
        self.max_attempts_before_hint = 2
        self.max_hints_before_tell = settings.MAX_HINTS
        self.frustration_threshold = 3
```

Метод выбора педагогической стратегии:

```
def _select_strategy(self, situation, session):
    """Select tutoring strategy based on situation."""

    # Student solved it!
    if situation.get("student_state") == "solved":
        return "encourage"

    # Student is frustrated - be supportive
    if situation.get("emotional_state") == "frustrated":
        if session.hints_used < self.max_hints_before_tell:
            return "hint"
        else:
            return "tell" # Last resort

    # Student made an error
    if situation.get("student_state") == "made_error":
```

```

        return "rectify"

    # Student is asking a question
    if situation.get("is_asking_question"):
        return "problematize"

    # Too many attempts without progress
    if session.attempts > self.max_attempts_before_hint:
        if session.hints_used < self.max_hints_before_tell:
            return "hint"
        elif session.hints_used >= self.max_hints_before_tell:
            return "tell"

    # Default: Guide with questions
    return "scaffolding"

```

Механизм предотвращения раскрытия ответа:

```

def _is_revealing_answer(self, message, task):
    """Check if response accidentally reveals the answer."""
    answer_str = str(task.answer).lower().strip()
    message_lower = message.lower()

    if len(answer_str) < 3:
        return False

    if answer_str in message_lower:
        return True

    revealing_patterns = [
        f"the answer is", f"solution is",
        f"equals {answer_str}", f"= {answer_str}",
    ]

    return any(p in message_lower for p in revealing_patterns)

def _sanitize_response(self, response, task):
    """Sanitize response that might reveal the answer."""
    safe_message = (
        "Let's think about this step by step. "
        "What approach have you tried so far?"
    )
    return TutorResponse(
        move=TutorMove.SCAFFOLDING,
        message=safe_message,
        is_telling=False
    )

```

ПРИЛОЖЕНИЕ Б

Исходный код модуля выполнения кода (фрагменты)

Класс CodeSecurityChecker — проверка безопасности:

```
FORBIDDEN_IMPORTS = {
    'os', 'sys', 'subprocess', 'socket', 'requests',
    'urllib', 'http', 'ctypes', 'multiprocessing',
    'pickle', 'shutil', 'importlib'
}

FORBIDDEN_BUILTINS = {
    '__import__', 'exec', 'eval', 'compile', 'open',
    'breakpoint', 'exit', 'quit'
}

class CodeSecurityChecker:
    @staticmethod
    def check_code(code: str) -> Tuple[bool, str]:
        try:
            tree = ast.parse(code)
        except SyntaxError as e:
            return False, f"Syntax error: line {e.lineno}"

        for node in ast.walk(tree):
            if isinstance(node, ast.Import):
                for alias in node.names:
                    module = alias.name.split('.')[0]
                    if module in FORBIDDEN_IMPORTS:
                        return False, f"Forbidden import: {module}"

            elif isinstance(node, ast.Call):
                if isinstance(node.func, ast.Name):
                    if node.func.id in FORBIDDEN_BUILTINS:
                        return False, f"Forbidden: {node.func.id}"

        return True, ""
```

Класс CodeExecutor — выполнение кода в песочнице:

```
class CodeExecutor:
    def __init__(self, timeout_seconds=5.0, memory_limit_mb=128):
        self.timeout = timeout_seconds
        self.memory_limit = memory_limit_mb
        self.security_checker = CodeSecurityChecker()
```



```
def execute(self, code: str, input_data: str = ""):
    # Security check
    is_safe, error = self.security_checker.check_code(code)
    if not is_safe:
        return ExecutionResult(
            status=ExecutionStatus.SECURITY_ERROR,
            error=error
        )

    # Create temp file
    with tempfile.NamedTemporaryFile(
        mode='w', suffix='.py', delete=False
    ) as f:
        f.write(code)
        temp_file = f.name

    try:
        start_time = time.time()
        result = subprocess.run(
            [sys.executable, temp_file],
            input=input_data,
            capture_output=True,
            text=True,
            timeout=self.timeout
        )
        exec_time = (time.time() - start_time) * 1000

        if result.returncode == 0:
            return ExecutionResult(
                status=ExecutionStatus.SUCCESS,
                output=result.stdout.strip(),
                execution_time_ms=exec_time
            )
        else:
            return ExecutionResult(
                status=ExecutionStatus.RUNTIME_ERROR,
                error=result.stderr.strip()
            )
    except subprocess.TimeoutExpired:
        return ExecutionResult(
            status=ExecutionStatus.TIME_LIMIT,
            error=f"Timeout ({self.timeout}s)"
        )
    finally:
        os.unlink(temp_file)
```

Класс **ErrorAnalyzer** — анализ ошибок для обратной связи:

```
class ErrorAnalyzer:
    ERROR_PATTERNS = {
        "ZeroDivisionError": {
            "ru": "Деление на ноль",
            "hint": "Проверьте делитель",
            "example": "if divisor != 0: result = a / divisor"
        },
        "IndexError": {
```

```

        "ru": "Выход за границы",
        "hint": "Проверьте длину списка",
        "example": "if 0 <= i < len(arr): value = arr[i]"
    },
    "RecursionError": {
        "ru": "Бесконечная рекурсия",
        "hint": "Проверьте базовый случай",
        "example": "if n <= 1: return 1"
    }
}

@classmethod
def analyze(cls, error: str) -> Dict[str, str]:
    for pattern, advice in cls.ERROR_PATTERNS.items():
        if pattern in error:
            return {
                "type": advice["ru"],
                "hint": advice["hint"],
                "example": advice["example"]
            }
    return {"type": "Ошибка выполнения", "hint": "Проверьте код"}

```